
Practica semana 9

Tema: Manejo Avanzado de excepciones, logging y monitoreo en tiempo real

OBJETIVO

- Implementar jerarquías de excepciones personalizadas en C++.
- Integrar un sistema de logging para registrar eventos críticos y de depuración.
- Simular un monitoreo en tiempo real con registro de errores y métricas del sistema.

MARCO TEÓRICO

1. **Excepciones personalizadas:** permiten representar errores específicos con mayor semántica que `std::exception`.
2. **Logging:** es el registro estructurado de eventos que ayuda a la depuración, análisis de errores y auditoría. En C++ se puede implementar con librerías como `spdlog` o de forma manual con archivos de texto.
3. **Monitoreo en tiempo real:** implica capturar métricas y estados de ejecución durante la operación del sistema.

Desarrollo

- a. Compila y ejecuta el programa.
- b. Revisar que el archivo `system.log` se haya creado con los errores registrados
- c. Simula distintos errores (división entre cero, valores negativos, entradas no numéricas).
- d. Implementa un ciclo que divida una lista de números y registre los errores de forma continua (monitoreo en tiempo real).

CODIGO C++

```
#include <iostream>
#include <fstream>
#include <stdexcept>
#include <ctime>
#include <string>

// EXCEPCIONES PERSONALIZADAS
class DivisionByZeroException : public std::runtime_error {
public:
    DivisionByZeroException()
        : std::runtime_error("Error: División entre cero.") {}
};

class NegativeValueException : public std::runtime_error {
public:
    NegativeValueException()
        : std::runtime_error("Error: Valor negativo detectado.") {}
};

class InvalidInputException : public std::runtime_error {
public:
    InvalidInputException()
        : std::runtime_error("Error: Entrada no numérica.") {}
};
```

```

// LOGGER
class Logger {
private:
    std::ofstream logfile;

public:
    Logger(const std::string& filename) {
        logfile.open(filename, std::ios::app);
        if (!logfile.is_open()) {
            throw std::runtime_error(
                "No se pudo abrir el archivo log."
            );
        }
    }
    ~Logger() {
        if (logfile.is_open()) {
            logfile.close();
        }
    }
    void log(const std::string& message) {
        time_t now = time(0);
        std::string fecha = ctime(&now);
        // Elimina el salto de línea de ctime()
        fecha.pop_back();
        logfile << "[" << fecha << "]" "
            << message << std::endl;
    }
};

// FUNCIÓN DIVIDIR
double dividir(double a, double b) {
    // División entre cero
    if (b == 0) {
        throw DivisionByZeroException();
    }
    // Valores negativos
    if (a < 0 || b < 0) {
        throw NegativeValueException();
    }
    return a / b;
}

// FUNCIÓN PARA VALIDAR TEXTO
void validarEntrada(const std::string& texto) {
    bool esNumero = true;
    for (char c : texto) {
        if (!isdigit(c) && c != '.') {
            esNumero = false;
            break;
        }
    }
    if (!esNumero) {
        throw InvalidInputException();
    }
}

int main() {
    Logger logger("system.log");

```

```

// PRUEBA 1
try {
    std::cout << "Prueba 1 -> 10 / 2"
                << std::endl;
    double resultado = dividir(10, 2);
    std::cout << "Resultado: "
                << resultado
                << std::endl;
    logger.log("10 / 2 ejecutado correctamente.");
}
catch (const std::exception& ex) {
    std::cerr << ex.what() << std::endl;
    logger.log(ex.what());
}
std::cout << std::endl;

// PRUEBA 2
try {

    std::cout << "Prueba 2 -> 10 / 0"
                << std::endl;
    double resultado = dividir(10, 0);
    std::cout << "Resultado: "
                << resultado
                << std::endl;
}
catch (const std::exception& ex) {
    std::cerr << ex.what() << std::endl;
    logger.log(ex.what());
}
std::cout << std::endl;

// PRUEBA 3
try {
    std::cout << "Prueba 3 -> -8 / 2"
                << std::endl;
    double resultado = dividir(-8, 2);
    std::cout << "Resultado: "
                << resultado
                << std::endl;
}
catch (const std::exception& ex) {
    std::cerr << ex.what() << std::endl;
    logger.log(ex.what());
}
std::cout << std::endl;

// PRUEBA 4
try {
    std::cout << "Prueba 4 -> 20 / 5"
                << std::endl;
    double resultado = dividir(20, 5);
    std::cout << "Resultado: "
                << resultado
                << std::endl;
    logger.log("20 / 5 ejecutado correctamente.");
}
catch (const std::exception& ex) {
    std::cerr << ex.what() << std::endl;
    logger.log(ex.what());
}

```

```

std::cout << std::endl;
logger.log("Programa finalizado.");

return 0;
}

```

<https://www.onlinegdb.com/edit/bY34hSffK>

SALIDA ESPERADA

```

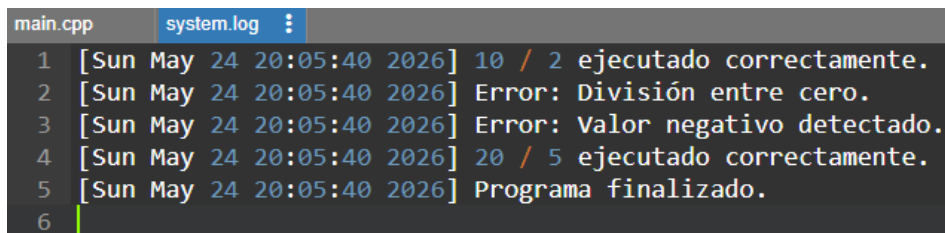
Prueba 1 -> 10 / 2
Resultado: 5

Prueba 2 -> 10 / 0
Error: División entre cero.

Prueba 3 -> -8 / 2
Error: Valor negativo detectado.

Prueba 4 -> 20 / 5
Resultado: 4

```



```

main.cpp system.log
1 [Sun May 24 20:05:40 2026] 10 / 2 ejecutado correctamente.
2 [Sun May 24 20:05:40 2026] Error: División entre cero.
3 [Sun May 24 20:05:40 2026] Error: Valor negativo detectado.
4 [Sun May 24 20:05:40 2026] 20 / 5 ejecutado correctamente.
5 [Sun May 24 20:05:40 2026] Programa finalizado.
6

```

IMPLEMENTA UN CICLO QUE DIVIDA UNA LISTA DE NÚMEROS Y REGISTRE LOS ERRORES DE FORMA CONTINUA (MONITOREO EN TIEMPO REAL).

```

#include <iostream>
#include <fstream>
#include <stdexcept>
#include <ctime>
#include <string>

class DivisionByZeroException : public std::runtime_error {
public:
    DivisionByZeroException()
        : std::runtime_error(
            "Error: División entre cero."
        ) {}
};

class NegativeValueException : public std::runtime_error {
public:
    NegativeValueException()
        : std::runtime_error(
            "Error: Valor negativo detectado."
        ) {}
};

```

```

class Logger {
private:
    std::ofstream logfile;

public:
    Logger(const std::string& filename) {
        logfile.open(
            filename,
            std::ios::app
        );
        if (!logfile.is_open()) {
            throw std::runtime_error(
                "No se pudo abrir el archivo log."
            );
        }
    }
    ~Logger() {
        if (logfile.is_open()) {
            logfile.close();
        }
    }
    void log(const std::string& message) {
        time_t now = time(0);
        std::string fecha = ctime(&now);
        // Elimina el salto de línea
        fecha.pop_back();
        logfile << "[" << fecha << "]" "
                << message
                << std::endl;
    }
};

double dividir(double a, double b) {
    // División entre cero
    if (b == 0) {
        throw DivisionByZeroException();
    }
    // Valores negativos
    if (a < 0 || b < 0) {
        throw NegativeValueException();
    }
    return a / b;
}

int main() {

    Logger logger("system.log");

    // Lista de divisiones
    double numeradores[] = {10, 20, -5, 30, 100};
    double denominadores[] = {2, 0, 5, -3, 4};
    int total = 5;

    std::cout << "==== MONITOREO EN TIEMPO REAL ====="
                << std::endl << std::endl;

    // Ciclo de monitoreo

```

```

for (int i = 0; i < total; i++) {
    try {
        std::cout << "Operacion "
                    << (i + 1)
                    << " -> "
                    << numeradores[i]
                    << " / "
                    << denominadores[i]
                    << std::endl;

        double resultado = dividir(
            numeradores[i],
            denominadores[i]
        );
        std::cout << "Resultado: "
                    << resultado
                    << std::endl;
        logger.log(
            "Operacion correcta: " +
            std::to_string(numeradores[i]) +
            " / " +
            std::to_string(denominadores[i]) +
            " = " +
            std::to_string(resultado)
        );
    }

    // División entre cero
    catch (const DivisionByZeroException& ex) {
        std::cerr << ex.what()
                    << std::endl;
        logger.log(
            std::string("ERROR -> ") +
            ex.what()
        );
    }

    // Valores negativos
    catch (const NegativeValueException& ex) {
        std::cerr << ex.what()
                    << std::endl;
        logger.log(
            std::string("ERROR -> ") +
            ex.what()
        );
    }

    // Cualquier otra excepción
    catch (const std::exception& ex) {
        std::cerr << "Excepcion general: "
                    << ex.what()
                    << std::endl;
        logger.log(
            std::string("Excepcion general: ") +
            ex.what()
        );
    }
}

```

```

        std::cout << "-----"
        << std::endl;
    }
    logger.log("Monitoreo finalizado.");
    std::cout << std::endl
        << "Revisa el archivo system.log"
        << " para los registros."
        << std::endl;

    return 0;
}

```

https://www.onlinegdb.com/edit/J_ilj9jxL

SALIDA ESPERADA

```

===== MONITOREO EN TIEMPO REAL =====

Operacion 1 -> 10 / 2
Resultado: 5
-----
Operacion 2 -> 20 / 0
Error: División entre cero.
-----
Operacion 3 -> -5 / 5
Error: Valor negativo detectado.
-----
Operacion 4 -> 30 / -3
Error: Valor negativo detectado.
-----
Operacion 5 -> 100 / 4
Resultado: 25
-----

Revisa el archivo system.log para los registros.

```

main.cpp	system.log
1	[Sun May 24 20:25:45 2026] Operacion correcta: 10.000000 / 2.000000 = 5.000000
2	[Sun May 24 20:25:45 2026] ERROR -> Error: División entre cero.
3	[Sun May 24 20:25:45 2026] ERROR -> Error: Valor negativo detectado.
4	[Sun May 24 20:25:45 2026] ERROR -> Error: Valor negativo detectado.
5	[Sun May 24 20:25:45 2026] Operacion correcta: 100.000000 / 4.000000 = 25.000000
6	[Sun May 24 20:25:45 2026] Monitoreo finalizado.
7	